

OSSP (Operating Systems And Systems Programming) - 25CS2104E

Roll No. : 2520030029

Name : S. Pujith Pavan Kumar

SEC-8

PRACTICAL-1:

Aim: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using fork(). 3. Execute the command in the child process using an appropriate exec() system call. 4. Allow the parent process to wait for the child using wait (). 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (uname, lscpu, lsblk, ps, top), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

Procedure :

```
siddinapujith07@DESKTOP-FdHCAIT: X + | v
siddinapujith07@DESKTOP-FdHCAIT:~$ mkdir OSSP
mkdir: OSSP: File exists
siddinapujith07@DESKTOP-FdHCAIT:~$ cd OSSP
siddinapujith07@DESKTOP-FdHCAIT:~/OSSP$ nano command_exec.c
siddinapujith07@DESKTOP-FdHCAIT:~/OSSP$ gcc command_exec.c -o command_exec
siddinapujith07@DESKTOP-FdHCAIT:~/OSSP$ ls
command_exec  command_exec.c  command_exec.c.save
siddinapujith07@DESKTOP-FdHCAIT:~/OSSP$ ./command_exec
Enter a Linux command: uname

Parent Process
Parent PID: 3669

Child Process
Child PID: 3670
Parent PID: 3669
Linux
Child process completed.
siddinapujith07@DESKTOP-FdHCAIT:~/OSSP$ ./command_exec
Enter a Linux command: lscpu

Parent Process
Parent PID: 3680

Child Process
Child PID: 3681
Parent PID: 3680
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 12
On-line CPU(s) list:   0-11
Vendor ID:              GenuineIntel
Model name:              13th Gen Intel(R) Core(TM) i5-1335
CPU family:              6
Model:                  186
Thread(s) per core:     2
```

```

Model: 186
Thread(s) per core: 2
Core(s) per socket: 6
Socket(s): 1
Stepping: 3
BogoMIPS: 4991.99
Flags: fpu vme de pse tsc msr pae mce cx8
      apic sep mtrr pge mca cmov pat ps
      e36 clflush mmx fxsr sse sse2 ss h
      t syscall nx pdpe1gb rdtscp lm con
      stant_tsc rep_good nopl xtopology
      tsc_reliable nonstop_tsc cpuid tsc
      _known_freq pni pclmuldq vmx ssse
      3 fma cx16 pcid sse4_1 sse4_2 x2ap
      ic movbe popcnt tsc_deadline_timer
      aes xsave avx f16c rdrand hypervi
      sor lahf_lm abm 3dnowprefetch ssbd
      ibrs ibpb stibp ibrs_enhanced tpr
      _shadow ept vpid ept_ad fsgsbase t
      sc_adjust bmi1 avx2 smep bmi2 erms
      invpcid rdseed adx smap clflushop
      t clwb sha_ni xsaveopt xsavec xget
      bvl xsaves avx_vnni vnni umip wait
      pkg gfni vaes vpclmuldq rdpid mov
      dir movdir64b fsrm md_clear seria
      lize ibt flush_lld arch_capabiliti
      es

Virtualization features:
Virtualization: VT-x
Hypervisor vendor: Microsoft
Virtualization type: full
Caches (sum of all):
L1d: 288 KiB (6 instances)
L1i: 192 KiB (6 instances)
L2: 7.5 MiB (6 instances)
L3: 12 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0-11
Vulnerabilities:

```

```

Vulnerabilities:
Gather data sampling: Not affected
Ghostwrite: Not affected
Indirect target selection: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Mitigation; Clear Register File
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypa
                  ss disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barrie
             rs and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic I
             BRS; IBPB conditional; PBRSSB-eIBRS
             SW sequence; BHI BHI_DIS_S
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected

```

```

Child process completed.
venkatsaikrishna5@VSK-HP:~/OSSP$ ./command_exec
Enter a Linux command: lsblk

```

```

Parent Process
Parent PID: 3693

```

```

Child Process
Child PID: 3694
Parent PID: 3693
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 356.9M 1 disk
sdb 8:16 0 159.4M 1 disk
sdc 8:32 0 2G 0 disk [SWAP]
sdd 8:48 0 1T 0 disk /mnt/wslg/distro
/

```

```
Child process completed.
siddinapujith07@DESKTOP-FdHCAIT: ~/OSSP$ ps
Parent Process
Child Process
Parent PID: 3704
Child PID: 3705
Parent PID: 3704
  PID TTY          TIME CMD
 3578 pts/0        00:00:00 bash
 3704 pts/0        00:00:00 command_exec
 3705 pts/0        00:00:00 ps
Child process completed.
siddinapujith07@DESKTOP-FdHCAIT:~/OSSP$ top
top - 06:04:56 up 52 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7796.6 total, 7001.0 free, 643.8 used, 31.8 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,  0.0 used, 712.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
    1 root        20   0   24144   14448   10604 S   0.0   0.2   0:01.75 systemd
    2 root        20   0   3180    2204   2072 S   0.0   0.0   0:00.03 init-systemd(CUb
    6 root        20   0   3180    2048   1940 S   0.0   0.0   0:00.00 init
   49 root        19  -1   50400   16812   15524 S   0.0   0.2   0:00.39 systemd-journal
   84 systemd+    20   0   22540   14628   12012 S   0.0   0.2   0:00.14 systemd-resolve
   93 root        20   0   35360   12352   9268 S   0.0   0.2   0:02.23 systemd-udev
  113 root        20   0   2888    1948   1820 S   0.0   0.0   0:00.06 chronyd-starter
  114 root        20   0   4472    3104   2860 S   0.0   0.0   0:00.02 cron
  116 message+   20   0   8792    5492   4732 S   0.0   0.1   0:00.18 dbus-daemon
```

```
top - 06:05:05 up 52 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 7796.6 total, 7012.0 free, 632.9 used, 311.8 buff/cache
MiB Swap: 2048.0 total, 2048.0 free,  0.0 used, 7163.8 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
 3577 root        20   0   3200    1256   1108 S   0.0   0.0   0:00.05 Relay(3578)
    1 root        20   0   24144   14448   10604 S   0.0   0.2   0:01.75 systemd
    2 root        20   0   3180    2204   2072 S   0.0   0.0   0:00.03 init-systemd(Ub
    6 root        20   0   3180    2048   1940 S   0.0   0.0   0:00.00 init
   49 root        19  -1   50400   16812   15524 S   0.0   0.2   0:00.39 systemd-journal
   84 systemd+    20   0   22540   14628   12012 S   0.0   0.2   0:00.14 systemd-resolve
   93 root        20   0   35360   12352   9268 S   0.0   0.2   0:02.23 systemd-udev
  113 root        20   0   2888    1948   1820 S   0.0   0.0   0:00.06 chronyd-starter
  114 root        20   0   4472    3104   2860 S   0.0   0.0   0:00.02 cron
  116 message+   20   0   8792    5492   4732 S   0.0   0.1   0:00.18 dbus-daemon
  159 root        20   0   44092   29780   14648 S   0.0   0.4   0:00.40 networkd-dispat
  163 root        20   0   18492   9528    8224 S   0.0   0.1   0:00.21 systemd-logind
  177 syslog      20   0  220548   5892    4624 S   0.0   0.1   0:00.12 rsyslogd
  216 root        20   0   5492    2712   2528 S   0.0   0.0   0:00.01 agetty
  217 _chrony      20   0   20420   6384    5608 S   0.0   0.1   0:00.14 chronyd
  232 _chrony      20   0   12092   2448    1804 S   0.0   0.0   0:00.00 chronyd
  246 root        20   0  123456  32620   18004 S   0.0   0.4   0:00.34 unattended-upgr
  318 root        20   0   8648    5516   4672 S   0.0   0.1   0:00.02 login
  384 venkats+    20   0   21936   11380   9176 S   0.0   0.1   0:00.37 systemd
  386 venkats+    20   0  22012   4112   2112 S   0.0   0.1   0:00.00 (sd-pam)
  412 venkats+    20   0   6136    5416   3864 S   0.0   0.1   0:00.04 bash
  828 polkitd     20   0  306444   8444    7628 S   0.0   0.1   0:00.21 polkitd
 3576 root        20   0   3184    1116    980 S   0.0   0.0   0:00.00 SessionLeader
 3578 venkats+    20   0   6268    5544   3872 S   0.0   0.1   0:00.11 bash
 3715 venkats+    20   0   2768    1820   1720 S   0.0   0.0   0:00.00 command_exec
 3716 venkats+    20   0   8172    6052   3948 R   0.0   0.1   0:00.03 top
```

Code:

```
GNU nano 8.7.1                                     command_exec.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    char command[100];

    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        // Executes only if exec fails
        printf("Invalid command!\n");
        exit(1);
    }
    else {
        // Parent Process
        printf("\nParent Process\n");

        ^G Help      ^O Write Out    ^F Where Is     ^K Cut          ^T Execute      ^C Location     M-U Undo        M-A Set Mark
        ^X Exit      ^R Read File    ^\ Replace      ^U Paste        ^J Justify      ^_ Go To Line   M-E Redo        M-G Copy

GNU nano 8.7.1                                     command_exec.c
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        // Child Process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        // Executes only if exec fails
        printf("Invalid command!\n");
        exit(1);
    }
    else {
        // Parent Process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

Observation :

- The parent and child processes were successfully created using `fork()`, and their PIDs were displayed.
- The child successfully executed the user-entered Linux command using `exec()`.
- The parent process waited for the child using `wait()`.
- `uname`, `lscpu`, and `lsblk` displayed operating system, CPU, and storage information.
- `ps` and `top` displayed running processes and resource usage.
- The experiment showed that the operating system manages hardware resources and provides services that allow applications to use CPU, memory, storage, and I/O devices without directly accessing the hardware.

Result :

The experiment successfully demonstrated **Linux command execution using `fork()`, `exec()`, and `wait()`** and investigated the relationship between hardware resources and OS services. The Linux operating system was observed to provide effective abstractions and management of **CPU, memory, storage, processes, and I/O resources**.